MyAnimLab

Analyse d'assurance qualité du logiciel

Revue initiale : commit `a2b130b` — Suivi après fusion : `main` au commit `4bcdec1`

18 octobre 2026

1 Objet, périmètre et méthode

MyAnimLab est une application web PHP/MySQL (dérivée de MyVivarium) de gestion d'animalerie : registre d'entrées et de sorties, cages, lignées, essais, protocoles et expérimentations, grilles cliniques et points limites, autorisations éthiques, pharmacie et rapport annuel d'utilisation des animaux. Ces données relèvent de la directive 2010/63/UE et de sa transposition française (articles R.214-87 et suivants du Code rural, arrêtés du 1er février 2013). Leur exactitude et leur traçabilité sont donc une exigence réglementaire, au-delà de la simple qualité logicielle.

Périmètre. 101 scripts PHP à la racine, `lang/`, `tools/`, `setup/`, `docker/`, le schéma `database/schema.sql` et 26 migrations (environ 26 400 lignes PHP et 1 900 lignes SQL).

Méthode. Revue statique du code et de la configuration, contrôle syntaxique (`php -l`) de tous les fichiers, recherches ciblées (requêtes SQL construites par concaténation, contrôles d'authentification et de jeton CSRF, téléversements, journalisation), vérification de la cohérence entre le schéma et les migrations, mesure de la couverture du catalogue de traduction. L'application n'a pas été exécutée : il n'y a pas eu de test dynamique ni de test d'intrusion.

2 Synthèse

Domaine	Constat principal	Évaluation
Sécurité	Fichier `.env` et téléversements servis par Apache ; quatre points d'entrée sans authentification	Insuffisant
Intégrité des données	Modifier un suivi expérimental écrase les valeurs précédentes sans en garder trace	À renforcer
Vérification	Aucun test automatisé, aucune intégration continue	Insuffisant
Gestion de configuration	Migrations sans outil d'application ; la procédure d'installation déploie le dépôt amont	À renforcer
Maintenabilité	Fichiers monolithiques, doublons `hc_*`/`bc_*`, contrôle CSRF recopié dans chaque page	À renforcer
Accès aux données SQL	357 requêtes préparées ; les rares interpolations portent sur des entiers	Satisfaisant
Authentification	bcrypt, verrouillage après 3 échecs, régénération de l'identifiant de session	Satisfaisant
Internationalisation	1 495 entrées ; tous les libellés `tr()` sont traduits	Satisfaisant

Bilan : 3 constats critiques, 6 majeurs et 5 mineurs. Les constats critiques sont des failles de sécurité exploitables dès le déploiement par `setup/setup.sh`. Ils doivent être corrigés avant toute mise en production.

3 Suivi après fusion (`main`, commit `4bcdec1`)

Ce suivi porte sur les commits fusionnés depuis la revue initiale, en particulier `25a28f6` (corrections de contrôle d'accès, de CSRF, de SQL et de XSS) et `fb5877e` (tests automatisés et généralisation de `session_config.php`). Chaque constat a été vérifié de nouveau dans le code. Les 149 tests unitaires passent en local. Les 45 tests d'intégration et HTTP nécessitent une base de données : ils ont été ignorés en local mais s'exécutent dans la CI, qui est verte sur `main`.

Constat	Statut	Constat de vérification
QA-01	Ouvert	Toujours aucun `.htaccess` ; `DocumentRoot` reste la racine du dépôt, donc `.env` reste servi.
QA-02	Corrigé	`notes_lib.php` exige une session, une requête POST, le jeton CSRF et l'auteur de la note ou un administrateur. `get_task.php` et `get_reminder.php` exigent une session ; une tâche n'est renvoyée qu'aux personnes concernées. `manage_strain.php` et `manage_animal_lines.php`, qui étaient eux aussi ouverts, sont désormais réservés aux administrateurs.
QA-03	Partiel	Le SVG est refusé. En revanche, les fichiers restent servis depuis `uploads/` sous leur nom d'origine, sans contrôle d'accès, avec les droits `0777` et sans contrôle du type MIME.
QA-04	Partiel	Les corrections d'actes expérimentaux conservent la valeur initiale et exigent un motif (bonne pratique). Le suivi quotidien est en revanche toujours écrasé (`experiment_followup.php`, l. 247–274).
QA-05	Ouvert	Aucun changement ; `setup.sh` accorde toujours `GRANT ALL` au compte de l'application.
QA-06	Largement corrigé	PHPUnit 11 : 194 tests unitaires, d'intégration et HTTP, avec une CI GitHub Actions sur MariaDB. Il reste à ajouter l'analyse statique et `composer audit`.
QA-07	Partiel	Un test vérifie que les migrations (désormais 34) sont numérotées sans trou. Il n'y a toujours pas d'outil pour les appliquer, et `REPO_URL` pointe toujours vers le dépôt amont.
QA-08	Ouvert	`process_reminders.php` n'a toujours pas de garde CLI.
QA-09	Ouvert	`export_for_v2.php` exporte toujours la table `users` entière.
QA-10	Partiel	Les contrôles des notes sont centralisés dans `notes_lib.php`, mais le contrôle CSRF reste recopié dans chaque page.
QA-11	Ouvert	`nt_add.php` et `nt_rmv.php` renvoient toujours le texte de l'erreur SQL au client.
QA-12	Stable	Les SQL de `manage_tasks.php` sont désormais paramétrés, mais le nouveau code ajoute des interpolations (`terrarium_lib.php`, `terrarium_view.php`). Toutes restent converties en entiers (28 occurrences).
QA-13	Ouvert	`America/New_York` est toujours la valeur par défaut, désormais aussi dans `manage_tasks.php`.
QA-14	Partiel	`.phpunit.cache` est ignoré et `require-dev` est déclaré. `images/.DS_Store` et `uploads/` ne sont toujours pas traités, et la version de PHP n'est pas déclarée.

Nouveaux points positifs. Double authentification TOTP (`auth_lib.php`, `login_2fa.php`) avec protection contre le rejeu (`totp_last_step`), limite de 5 essais suivie d'un verrouillage de 15 minutes, codes de secours, et obligation de configuration appliquée par `session_config.php`. Toutes les pages ouvrent désormais la session par `session_config.php`, avec une durée d'inactivité réglable, et un test automatique le vérifie. Le registre des stupéfiants et le livret de formation renforcent la traçabilité réglementaire.

Bilan du suivi. Sur les 3 constats critiques, 1 est corrigé (QA-02), 1 est partiel (QA-03) et 1 reste ouvert (QA-01, le plus grave). Le socle de vérification (QA-06) est désormais en place. Les priorités restantes sont : QA-01, la fin de QA-03, QA-08, puis le versionnement des suivis quotidiens (QA-04).

4 Constats détaillés (revue initiale)

4.1 Constats critiques

QA-01 — Secrets et fichiers internes servis par le serveur web

Gravité : **Critique**

Preuve `setup/setup.sh:149` définit `DocumentRoot` sur la racine du dépôt et le README demande d'y copier `.env`. Le dépôt ne contient aucun `.htaccess`, et la configuration Apache par défaut ne protège que les fichiers `.ht*`.

Risque Une simple requête `GET /.env` renvoie les identifiants MySQL et SMTP. `database/schema.sql`, `composer.lock`, `tools/` et `docs/` sont eux aussi publics.

Action Placer la racine web dans un sous-dossier `public/`, ou à défaut ajouter un `.htaccess` racine qui refuse `.env`, `*.sql`, `composer.*`, `tools/`, `database/`, `setup/` et `vendor/`. Vérifier ensuite chaque déploiement avec `curl -I https://`*<hôte>*`/.env`, qui doit répondre 403 ou 404.

QA-02 — Points d'entrée sans authentification

Gravité : **Critique**

Preuve `nt_edit.php` n'ouvre pas de session et modifie n'importe quelle note à partir de son identifiant. `nt_rmv.php` supprime n'importe quelle note sans vérifier la session ni l'auteur, alors que son en-tête affirme le contraire. `get_task.php` et `get_reminder.php` renvoient des données en JSON sans vérifier la session.

Risque Un visiteur anonyme peut altérer ou détruire des notes de cage (perte de traçabilité) et énumérer les tâches et rappels.

Action Créer un garde commun (`require_login()`, rôle et propriétaire), l'appliquer à ces quatre fichiers et ajouter le contrôle CSRF aux deux points d'entrée qui écrivent.

QA-03 — Téléversements de cage publics et SVG accepté

Gravité : **Critique**

Preuve `hc_edit.php`, l. 341–372 et `bc_edit.php`, l. 258–291 écrivent dans `uploads/`*<cage>*`/`*<nom d'origine>*, créé avec les droits `0777`. Les fichiers sont servis directement (`hc_view.php:507`). L'extension `svg` est autorisée et le type MIME réel n'est pas contrôlé.

Risque Un SVG contenant du JavaScript s'exécute dans l'origine de l'application (XSS stockée, vol de session). Comme les chemins sont prévisibles, les documents sont accessibles sans être connecté.

Action Reprendre le modèle de `ethics_lib.php` : stockage hors de la racine web ou derrière un `Require all denied`, nom aléatoire, contrôle par `finfo`, téléchargement par un script qui vérifie les droits. Retirer `svg` de la liste et passer les droits à `0750`.

4.2 Constats majeurs

QA-04 — Écrasement silencieux des suivis expérimentaux

Gravité : **Majeur**

Preuve `experiment_followup.php`, l. 215–242 : une modification fait un `ON DUPLICATE KEY UPDATE` sur `experiment_followups`, puis un `DELETE` et une réinsertion des scores et des observations. L'ancienne valeur n'est conservée nulle part et la modification n'est pas journalisée.

Risque Les principes ALCOA+ (données attribuables, originales, durables) ne sont plus respectés. Lors d'une inspection DDPP ou d'une revue de comité d'éthique, on ne peut pas reconstituer le score clinique qui a motivé, ou non, l'application d'un point limite.

Action Versionner les suivis (table d'historique, ou ligne « remplacée » avec un `superseded_by`), exiger un motif de modification et journaliser l'avant et l'après dans `activity_log`. Appliquer le même principe aux autres écrans de modification des données réglementaires (registre, sorties, prescriptions).

QA-05 — Journal d'audit modifiable et incomplet

Gravité : **Majeur**

Preuve La table `activity_log` (`schema.sql:792`) n'enregistre que du texte libre, sans valeurs avant ni après. Le compte MySQL de l'application peut y faire `UPDATE` et `DELETE`.

Action Créer un compte MySQL limité à `INSERT` et `SELECT` sur les tables d'audit, ajouter des champs structurés (`old_value`, `new_value`, motif) et, si besoin, un chaînage par empreinte des enregistrements pour rendre toute altération détectable.

QA-06 — Aucune vérification automatisée

Gravité : **Majeur**

Preuve Le dépôt ne contient ni répertoire de tests, ni PHPUnit dans `composer.json`, ni workflow dans `.github/`, ni outil d'analyse statique. Les 112 commits (dont 64 générés par un assistant) ont été intégrés sans contrôle automatique, et seules 2 pull requests ont été fusionnées.

Action Mettre en place un socle minimal :

1. une CI GitHub Actions avec `php -l`, PHPStan (niveau 5 au départ), `composer audit` et le chargement de `schema.sql` dans MySQL 8 ;
2. des tests PHPUnit des fonctions métier de `animal_lib.php` et `ethics_lib.php` (validité des autorisations, sévérité, calcul du rapport annuel) ;
3. quelques tests de bout en bout Playwright sur les parcours réglementaires (entrée d'un animal, suivi, point limite, sortie, rapport annuel) ;
4. une règle de protection de branche avec revue humaine obligatoire.

QA-07 — Gestion du schéma et du déploiement

Gravité : **Majeur**

Preuve Les 26 migrations de `database/migrations/` n'ont ni outil d'application ni table de version, et `schema.sql` est tenu à jour à la main en parallèle (il est cohérent à ce jour). Par ailleurs, `setup/setup.sh:4` et le README clonent `myvivarium/MyVivarium`, c'est-à-dire le dépôt amont et non ce fork.

Risque Une installation « en un clic » déploie une autre version du logiciel, et les mises à jour en production risquent d'oublier ou de rejouer des migrations.

Action Ajouter une table `schema_migrations` et un script `tools/migrate.php` idempotent. En CI, vérifier que l'application des migrations à partir de `001` produit le même schéma que `schema.sql`. Corriger les URL du README et de `setup.sh`.

QA-08 — Scripts cron accessibles par HTTP

Gravité : **Majeur**

Preuve `process_reminders.php` ne vérifie pas `PHP_SAPI`, alors que `daily_round_alert.php:14` le fait. Il suffit donc d'appeler son URL pour générer des tâches et remplir la file d'e-mails.

Action Ajouter le même garde CLI et, plus largement, sortir les scripts cron de la racine web.

QA-09 — Export complet contenant des secrets

Gravité : **Majeur**

Preuve `export_for_v2.php` exporte en JSON la table `users` entière, y compris les hachés de mots de passe et les jetons de réinitialisation. L'export est réservé aux administrateurs.

Action Exclure les colonnes sensibles, journaliser chaque export et limiter la durée de vie de la fonction à la période de migration.

4.3 Constats mineurs

QA-10 — Maintenabilité

Gravité : **Mineur**

Preuve Plusieurs fichiers dépassent 1 000 lignes (`hc_edit.php`, `bc_edit.php`) et les couples `hc_*` / `bc_*` sont presque identiques. Logique métier, SQL et HTML sont mêlés dans les mêmes fichiers. Le contrôle CSRF est recopié dans 44 fichiers sans fonction commune.

Action Factoriser progressivement : fonctions `csrf_field()` et `csrf_verify()`, repositories par entité, un seul écran paramétré pour les deux catégories de cages.

QA-11 — Fuite d'informations techniques

Gravité : **Mineur**

Preuve `nt_add.php` et `nt_rmv.php` renvoient `$con->error` et `$stmt->error` au client.

Action Journaliser l'erreur côté serveur et renvoyer un message générique.

QA-12 — Requêtes SQL interpolées

Gravité : **Mineur**

Preuve Une dizaine de requêtes interpolent des listes d'identifiants (`experiment_report.php:63`, `litter_manage.php:170`, etc.). Toutes les valeurs sont converties en entiers, donc sans risque aujourd'hui.

Action Utiliser partout des requêtes préparées, pour qu'une évolution future ne puisse pas introduire d'injection.

QA-13 — Valeurs par défaut inadaptées

Gravité : Mineur

Preuve Le fuseau horaire par défaut de `process_reminders.php` est `America/New_York`. Si le paramètre `timezone` est absent, les rappels et les tâches sont décalés.

Action Utiliser `Europe/Paris` par défaut, ou refuser de s'exécuter tant que le paramètre n'est pas défini.

QA-14 — Hygiène du dépôt

Gravité : Mineur

Preuve `images/.DS_Store` est versionné et `uploads/` n'est pas exclu dans `.gitignore`. Les contraintes de version de PHP et des extensions (`mysqli`, `fileinfo`) ne sont pas déclarées dans `composer.json`.

Action Nettoyer le dépôt, compléter `.gitignore`, déclarer `"php": ">=8.1"` et `ext-*` dans `composer.json`.

5 Points forts

— 357 requêtes préparées. Aucune injection SQL n'a été trouvée.
— Mots de passe hachés avec bcrypt, compte verrouillé 15 minutes après 3 échecs, identifiant de session régénéré à la connexion, cookies `HttpOnly`, `Secure` et `SameSite=Strict`.
— Documents d'éthique bien protégés (`ethics_lib.php`) : contrôle MIME, nom aléatoire, accès direct refusé, taille limitée.
— Suppression définitive d'un animal réservée aux administrateurs, confirmée par saisie de l'identifiant et journalisée avant suppression.
— Journaux métier spécialisés (événements et mouvements d'animaux, cycle de vie des cages, portées), transactions dans 18 scripts, 74 clés étrangères.
— Contrôle syntaxique réussi sur 100 % des fichiers PHP, catalogue de traduction complet, environnement Docker local isolé de la production.

6 Plan d'action priorisé

Priorité	Constats	Action	Effort
P0	QA-01, 02, 03, 08	Racine web dans `public/`, garde d'authentification commun, téléversements protégés, garde CLI	2–3 j
P1	QA-04, 05	Versionnement des suivis, motif obligatoire, audit avant et après non modifiable	3–5 j
P1	QA-06	CI (lint, PHPStan, `composer audit`, schéma), premiers tests PHPUnit, protection de branche	3 j
P2	QA-07, 09	Outil de migrations, URL du dépôt, export sans secrets	2 j
P3	QA-10 à 14	Factorisation, nettoyage, valeurs par défaut	continu

Pour un usage réglementaire, nous recommandons aussi de rédiger une courte documentation de validation : exigences, matrice de traçabilité entre exigences et tests, et procédure de gestion des changements. Elle sert de preuve de maîtrise du logiciel lors des inspections.

A Métriques relevées

Scripts PHP (racine)	101
Lignes PHP (y compris `lang/` et `tools/`)	≈ 26 400
Lignes SQL (schéma et migrations)	≈ 1 900
Tables du schéma / clés étrangères	55 / 74
Migrations	26
Requêtes préparées	357
Fichiers avec contrôle CSRF	44
Entrées du catalogue français / libellés manquants	1 495 / 0
Tests automatisés / workflows CI	0 / 0
Erreurs de syntaxe (`php -l`)	0